Complete programs accompanying chiral enumeration

Human Compression project

This appendix prints all three complete files accompanying the accepted chiral-enumeration edition. The reference enumeration constructs eleven finite models and checks the full marking search and its equivalence conventions. The table program evaluates the written formulas for all 32 source rows. The GAP program independently checks twenty of those involution counts. These finite checks validate the written proofs; they do not replace the arbitrary-group or arbitrary-rank argument. The original operational driver and the seven scoped Lean declarations remain separately inventoried. No scientific source file is shortened or altered here.

Reference enumeration and marking validation

`phase282_enumerate.py`

```
#!/usr/bin/env python3
"""Exact finite-table chiral search, with independent rotation-coordinate checks."""
import argparse, collections, hashlib, itertools, json, pathlib, time

def need(test, message):
    if not test: raise RuntimeError('PHASE282_REJECT ' + message)

def compose(a,b): return tuple(a[b[i]] for i in range(len(a)))
def cycle(n,seq):
    p=list(range(n))
    for a,b in zip(seq,seq[1:]+seq[:1]): p[a]=b
    return tuple(p)

class FiniteGroup:
    def __init__(self, generators):
        e=tuple(range(len(generators[0]))); el=[e]; pos={e:0}
        for x in el:
            for s in generators:
                y=compose(x,s)
                if y not in pos: pos[y]=len(el);el.append(y)
        self.perms=el;self.n=len(el)
        self.mul=[[pos[compose(x,y)] for y in el] for x in el]
        self.inv=[next(y for y in range(self.n) if self.mul[x][y]==0) for x in range(self.n)]
        self.orders=[]
        for x in range(self.n):
            y=x;k=1
            while y: y=self.mul[y][x];k+=1
            self.orders.append(k)
        self.cache={():1};self.full=(1<<self.n)-1
    def subgroup(self,gens):
        key=tuple(sorted(set(gens)-{0}))
        if key in self.cache:return self.cache[key]
        seen=1; todo=[0]
        for x in todo:
            for s in key:
                y=self.mul[x][s]
                if not (seen>>y)&1:seen|=1<<y;todo.append(y)
        self.cache[key]=seen;return seen
    def diff(self,a,b):return self.mul[self.inv[a]][b]
    def parabolics(self,alpha):
        # α0=1; all pair differences are generated by a star at the least index.
        result=[]
        for mask in range(1<<len(alpha)):
            ix=[i for i in range(len(alpha)) if (mask>>i)&1]
            result.append(self.subgroup([] if not ix else [self.diff(alpha[ix[0]],alpha[j]) for j in ix[1:]]))
```

```
        return result
    def ip(self,parabolics):
        return all((h&k)==parabolics[i&j] for i,h in enumerate(parabolics) for j,k in enumerate(parabolics))
    def canonical(self,gens):
        # The colored rooted Cayley graph uniquely determines a generating marking.
        names={0:0}; queue=[0];rows=[]
        for x in queue:
            row=[]
            for s in gens:
                y=self.mul[x][s]
                if y not in names:names[y]=len(names);queue.append(y)
                row.append(names[y])
            rows.extend(row)
        need(len(names)==self.n,'canonical marking generates input group')
        return tuple(rows)
    def conj_representatives(self):
        unseen=set(range(self.n));result=[]
        while unseen:
            x=min(unseen);result.append(x)
            unseen.difference_update(self.mul[self.mul[self.inv[g]][x]][g] for g in range(self.n))
        return result
    def flag_graph(self,alpha):
        # α0=1; r_i(g,+)=(g α_i,-), r_i(g,-)=(g α_i^-1,+).
        return [[2*self.mul[g][a]+1 for a in alpha] if sign==0 else
                [2*self.mul[g][self.inv[a]] for a in alpha]
                for g in range(self.n) for sign in (0,1)]

def graph_automorphisms(adj):
    # Every color-preserving automorphism of a connected deterministic graph
    # is uniquely forced by the image of vertex zero; try EVERY possible image.
    n=len(adj);accepted=[]
    for target in range(n):
        f={0:target};todo=[0];valid=True
        for x in todo:
            for y,z in zip(adj[x],adj[f[x]]):
                if y in f:
                    if f[y]!=z:valid=False;break
                else:f[y]=z;todo.append(y)
            if not valid:break
        if valid and len(f)==n and len(set(f.values()))==n:accepted.append(tuple(f[i] for i in range(n)))
    return accepted

def component(adj, root, colors):
    found={root};todo=[root]
    for v in todo:
        for i in colors:
            w=adj[v][i]
            if w not in found:found.add(w);todo.append(w)
    return found

def verify_flag_geometry(adj):
    rank=len(adj[0]);tests=0
    # Left G has exactly the two parity orbits, so these two roots exhaust
    # the component-intersection tests up to actual graph automorphisms.
    for root in (0,1):
        components=[component(adj,root,[i for i in range(rank) if mask>>i&1]) for mask in range(1<<rank)]
        for i in range(1<<rank):
            for j in range(1<<rank):
                need(components[i]&components[j]==components[i&j],'flag component intersection property')
                tests+=1
    return tests

def enumerate_group(G):
    start=time.monotonic();invol=[x for x,o in enumerate(G.orders) if o==2]
    reps={};stats=collections.Counter();max_depth=0
    def visit(alpha,old):
        nonlocal max_depth
        max_depth=max(max_depth,len(alpha));stats['admissible_prefixes']+=1
        if old[-1]==G.full:
            if len(alpha)>=3:
                key=G.canonical(alpha[1:]);reps.setdefault((len(alpha),key),alpha)
            return
        for x in invol:
            stats['tested_extensions']+=1
```

```
            if (old[-1]>>x)&1:continue
            if G.orders[G.diff(alpha[-1],x)]<=2:continue
            if any(G.orders[G.diff(a,x)]!=2 for a in alpha[:-1]):continue
            new=G.parabolics(alpha+[x])
            if G.ip(new):visit(alpha+[x],new)
    for a in G.conj_representatives():
        if G.orders[a]>2:
            alpha=[0,a];visit(alpha,G.parabolics(alpha))
    census=collections.Counter();oriented_chiral={};unrooted={};oriented_regular={}
    witnesses=[]
    for (rank,key),alpha in sorted(reps.items()):
        mirror=G.canonical([G.inv[a] for a in alpha[1:]])
        kind='directly_regular' if key==mirror else 'chiral'
        typ=tuple(G.orders[G.diff(a,b)] for a,b in zip(alpha,alpha[1:]))
        census[(rank,typ,kind)]+=1
        target=oriented_regular if kind=='directly_regular' else oriented_chiral
        target[(rank,key)]=alpha
        if kind=='chiral':unrooted.setdefault((rank,min(key,mirror)),alpha)
        adj=G.flag_graph(alpha);aut=graph_automorphisms(adj)
        need(len(aut)==G.n*(2 if kind=='directly_regular' else 1),'full colored flag automorphism group')
        need(all(adj[adj[v][i]][i]==v for v in range(2*G.n) for i in range(rank)),'flag adjacency involutions')
        need(all(adj[adj[v][i]][j]==adj[adj[v][j]][i] for v in range(2*G.n) for i in range(rank) for j in
        ↪  range(i+2,rank)),'distant flag adjacencies commute')
        component_tests=verify_flag_geometry(adj)
        mirrored=G.flag_graph([G.inv[a] for a in alpha])
        need(all((adj[v][i]^1)==mirrored[v^1][i] for v in range(2*G.n) for i in range(rank)), 'explicit mirror
        ↪  graph isomorphism')
        witnesses.append(dict(rank=rank,type=typ,kind=kind,alpha=alpha,automorphisms=len(aut),flag_orbits=2*G.
        ↪  n//len(aut),component_tests=component_tests))
    need(len(oriented_chiral)==2*len(unrooted),'mirror pairing is free on chiral orbits')
    # Independent rank-three coordinates: search ALL ordered σ pairs, not the
    # prefix recursion, and impose all three two-index subgroup intersections.
    rotation_keys=set();raw_rotation=0
    for s in range(G.n):
        if G.orders[s]<=2:continue
        for t in range(G.n):
            if G.orders[t]<=2 or G.orders[G.mul[s][t]]!=2:continue
            if G.subgroup([s,t])!=G.full:continue
            C=[G.subgroup([s]),G.subgroup([t]),G.subgroup([G.mul[s][t]])]
            if not all(C[i]&C[j]==1 for i in range(3) for j in range(i+1,3)):continue
            raw_rotation+=1;rotation_keys.add(G.canonical([s,G.mul[s][t]]))
    prefix3={key for (rank,key) in reps if rank==3}
    need(rotation_keys==prefix3,'independent full rank-three rotation enumeration')
    return dict(order=G.n,involutions=len(invol),max_prefix_rank=max_depth,stats=dict(stats),
        oriented_chiral=len(oriented_chiral),unrooted_chiral=len(unrooted),directly_regular=len(oriented_regul
        ↪  ar),
        census=[dict(rank=k[0],type=k[1],kind=k[2],oriented_count=v) for k,v in sorted(census.items())],
        rank3_rotation_raw=raw_rotation,rank3_rotation_orbits=len(rotation_keys),witnesses=witnesses,
        seconds=round(time.monotonic()-start,4))

def group_models():
    return [
        ('C1',[(0,)]),
        ('C2',[cycle(2,[0,1])]),
        ('C2xC2',[cycle(4,[0,1]),cycle(4,[2,3])]),
        ('C5', [cycle(5,[0,1,2,3,4])]),
        ('D10',[cycle(5,[0,1,2,3,4]),tuple((-x)%5 for x in range(5))]),
        ('A4',[cycle(4,[0,1,2]),cycle(4,[1,2,3])]),
        ('S4',[cycle(4,[0,1,2,3]),cycle(4,[0,1])]),
        ('A5',[cycle(5,[0,1,2]),cycle(5,[0,1,2,3,4])]),
        ('AGL1_5',[tuple((x+1)%5 for x in range(5)),tuple(2*x%5 for x in range(5))]),
        ('AGL1_7',[tuple((x+1)%7 for x in range(7)),tuple(3*x%7 for x in range(7))]),
        ('PSL2_7',[tuple((x+1)%7 if x<7 else 7 for x in range(8)),
                   tuple(7 if x==0 else 0 if x==7 else (-pow(x,-1,7))%7 for x in range(8))])]

def main():
    ap=argparse.ArgumentParser(description=__doc__);ap.add_argument('--out',type=pathlib.Path,required=True);a
    ↪  =ap.parse_args()
    need(not a.out.exists() and not a.out.is_symlink(),'fresh output')
    result={}
    for name,gens in group_models():
        G=FiniteGroup(gens);r=enumerate_group(G);result[name]=r
        print(name,json.dumps({k:r[k] for k in
        ↪  ['order','oriented_chiral','unrooted_chiral','directly_regular','seconds']}),flush=True)
```

```
    need(result['AGL1_5']['oriented_chiral']==2 and result['AGL1_5']['unrooted_chiral']==1,'small chiral torus
    ↪  mirror control')
    need(result['A4']['directly_regular']>0 and result['A4']['oriented_chiral']==0,'regular tetrahedron
    ↪  control')
    # Two structural negative controls use explicit independently checked models.
    A4=FiniteGroup(dict(group_models())['A4']);alpha=result['A4']['witnesses'][0]['alpha']
    inner=any(all(A4.mul[A4.mul[A4.inv[g]][a]][g]==A4.inv[a] for a in alpha) for g in range(A4.n))
    need(not inner,'tetrahedral inversion is outer in A4')
    A5=FiniteGroup(dict(group_models())['A5']);found=None
    invol=[i for i,o in enumerate(A5.orders) if o==2]
    for r in itertools.permutations(invol,3):
        if A5.mul[r[0]][r[2]]!=A5.mul[r[2]][r[0]] or A5.subgroup(r)!=A5.full:continue
        par=[A5.subgroup([r[i] for i in range(3) if mask>>i&1]) for mask in range(8)]
        if not A5.ip(par):continue
        rot=[0,A5.mul[r[0]][r[1]],A5.mul[r[0]][r[2]]]
        if A5.subgroup(rot)!=A5.full:continue
        if A5.ip(A5.parabolics(rot)):
            found=dict(markers=r,alpha=rot,type=[A5.orders[A5.mul[r[i]][r[i+1]]] for i in range(2)],
                       original_regular_flags=60,reconstructed_directly_regular_flags=120);break
    need(found is not None,'nonorientable regular A5 versus its orientable reconstruction')
    a.out.write_text(json.dumps(dict(status='PASS',models=result,controls=dict(
        inner_only_inversion_misclassifies_A4=True,mirror_quotient_changes_chiral_count=True,
        omitted_reflection_filter_includes_regular=True,nonorientable_regular=found)),indent=2)+'\n')
if __name__=='__main__':main()
```

Exact evaluation of all mathematical table rows

phase282_table_counts.py

```
#!/usr/bin/env python3
"""Exact mathematical columns of the revised BLT timing table; no timing claim."""
import argparse,json,math,pathlib
from fractions import Fraction

def need(ok,msg):
    if not ok:raise RuntimeError('PHASE282_REJECT '+msg)

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--out',type=pathlib.Path,required=True);a=ap.parse_args()
    need(not a.out.exists() and not a.out.is_symlink(),'fresh output')
    rows=[]
    for family,params in [('PSL3',[2,3,4,5,7,8,9,11,13,16,17,19]),('PSU3',[2,3,4,5,7,8,9,11]),('Alt',range(5,1
    ↪  1)),('Sym',range(5,11))]:
        for q in params:
            if family=='PSL3':
                order=q**3*(q**3-1)*(q*q-1)//math.gcd(3,q-1)
                inv=q*q*(q*q+q+1) if q%2 else (q**3-1)*(q+1)
            elif family=='PSU3':
                order=q**3*(q**3+1)*(q*q-1)//math.gcd(3,q+1)
                inv=q*q*(q*q-q+1) if q%2 else (q**3+1)*(q-1)
            else:
                order=math.factorial(q)//(2 if family=='Alt' else 1)
                inv=sum(math.factorial(q)//(2**k*math.factorial(k)*math.factorial(q-2*k))
                        for k in range(1,q//2+1) if family=='Sym' or k%2==0)
            ratio=Fraction(inv,order)
            rows.append(dict(family=family,q=q,order=order,involutions=inv,ratio=str(ratio),decimal=float(rati
            ↪  o)))
    need(len(rows)==32,'all revised source rows')
    need(next(r for r in rows if (r['family'],r['q'])==('PSL3',17))['involutions']==88723,'PSL3(17) exact
    ↪  count')
    a.out.write_text(json.dumps(dict(status='PASS',rows=rows,scope='Exact group orders and involution counts;
    ↪  neither original timings nor a chiral census.'),indent=2)+'\n')
    print(json.dumps(dict(status='PASS',rows=len(rows))))
if __name__=='__main__':main()
```

Independent GAP validation of involution counts

phase282_involution_check.g

```
Need:=function(ok,msg) if not ok then Error("PHASE282_REJECT ",msg);fi;end;;
for family in ["PSL","PSU"] do
 for q in [2,3,4,5] do
  if family="PSL" then
   G:=PSL(3,q); ord:=q^3*(q^3-1)*(q^2-1)/Gcd(3,q-1);
   if IsOddInt(q) then inv:=q^2*(q^2+q+1);else inv:=(q^3-1)*(q+1);fi;
  else
   G:=PSU(3,q);ord:=q^3*(q^3+1)*(q^2-1)/Gcd(3,q+1);
   if IsOddInt(q) then inv:=q^2*(q^2-q+1);else inv:=(q^3+1)*(q-1);fi;
  fi;
  actual:=Sum(Filtered(ConjugacyClasses(G),c->Order(Representative(c))=2),Size);
  Need(Size(G)=ord,"classical group order");Need(actual=inv,"classical involution count");
  Print("PHASE282_CLASSICAL_PASS ",family," ",q," order ",ord," involutions ",actual,"\n");
 od;
od;
for n in [5..10] do
 for family in ["Alt","Sym"] do
  if family="Alt" then G:=AlternatingGroup(n);else G:=SymmetricGroup(n);fi;
  inv:=Sum(Filtered([1..QuoInt(n,2)],k->family="Sym" or IsEvenInt(k)),
     k->Factorial(n)/(2^k*Factorial(k)*Factorial(n-2*k)));
  actual:=Sum(Filtered(ConjugacyClasses(G),c->Order(Representative(c))=2),Size);
  Need(actual=inv,"permutation involution count");
  Print("PHASE282_PERMUTATION_PASS ",family," ",n," involutions ",actual,"\n");
 od;
od;
Print("PHASE282_ALL_TWENTY_COUNTS_PASS\n");
QUIT;
```